

COMSATS University Islamabad

Department of Computer Science

Software Design Document

Quantum Virtual Machine (QVM)

A Hardware-Agnostic Quantum Circuit Execution Platform

Final Year Project

April 26, 2026

Contents

4	Software Design	1
4.1	Introduction	1
4.2	Design Methodology	1
4.3	System Overview	1
4.4	Architectural Design	3
4.5	Activity Diagrams	5
4.5.1	Circuit Execution Flow	5
4.5.2	Transpilation Process	7
4.5.3	Simulation Process	9
4.6	Class Diagram	9
4.7	Sequence Diagrams	12
4.7.1	CLI Execution Flow	12
4.7.2	API Request Flow	15
4.7.3	Transpilation Detail	16
4.8	State Transition Diagrams	18
4.9	Data Flow Diagrams	19
4.9.1	Level 1 DFD - System Level	19
4.9.2	Level 2 DFD - Module Level	19
4.10	Entity-Relationship Diagram	22
4.11	Data Dictionary	23
4.11.1	QuantumCircuit IR Structure	23
4.11.2	Operation Dictionary Structure	23
4.11.3	Condition Dictionary Structure	23
4.11.4	Classical Operation Dictionary Structure	24
4.11.5	Target Architecture Structure	24
4.11.6	Simulation Results Structure	24
4.12	Summary	25

List of Figures

4.1	System Context Diagram	2
4.2	System Architecture Diagram — 7-Layer Pipeline	4
4.3	Activity Diagram — Circuit Execution Flow	6
4.4	Activity Diagram — Transpilation Process	8
4.5	Activity Diagram — Simulation Process	10
4.6	Class Diagram — System Structure	11
4.7	Sequence Diagram — CLI Execution Flow	14
4.8	Sequence Diagram — REST API Request Flow	15
4.9	Sequence Diagram — Transpilation Detail	17
4.10	State Transition Diagram — Circuit Lifecycle	18
4.11	Data Flow Diagram — Level 1 (System Level)	20
4.12	Data Flow Diagram — Level 2 (Module Level)	21

List of Tables

4.1	QuantumCircuit Data Structure	23
4.2	Operation Data Structure	23
4.3	Condition Data Structure	24
4.4	Classical Operation Data Structure	24
4.5	TargetArchitecture Data Structure	24
4.6	Simulation Results Data Structure	25

Chapter 4

Software Design

4.1 Introduction

This chapter presents the software design of the Quantum Virtual Machine (QVM) system. The design follows object-oriented principles and emphasizes modularity, maintainability, and extensibility. The chapter documents the architectural decisions, design patterns, and structural organization that enable the QVM to achieve its goal of hardware-agnostic quantum circuit execution. The design is presented through multiple perspectives using UML diagrams, including architectural views, behavioral models, and data flow representations.

4.2 Design Methodology

The QVM system was designed using **Object-Oriented Design (OOD)** methodology with emphasis on the following principles:

Separation of Concerns: The system is divided into distinct modules (Parser, IR, Transpiler, Simulator, Visualizer) with well-defined responsibilities. Each module encapsulates a specific aspect of the quantum compilation and execution pipeline.

Pipeline Architecture: The design follows a linear pipeline pattern where data flows through sequential stages of transformation: Input $\rightarrow$ Parsing $\rightarrow$ IR $\rightarrow$ Transpilation $\rightarrow$ Simulation $\rightarrow$ Output. This architecture mirrors the natural workflow of quantum program execution.

Strategy Pattern: The transpiler module implements the Strategy pattern, allowing runtime selection between different routing algorithms (Greedy BFS, SABRE) without modifying client code.

Abstraction and Interfaces: The QuantumCircuit IR serves as an abstraction layer that decouples high-level circuit definitions from low-level execution details, enabling hardware independence.

Modularity and Extensibility: Each component is designed as an independent module with minimal coupling, allowing new parsers, simulators, or transpilation strategies to be added without affecting existing code.

4.3 System Overview

The QVM system architecture is organized as a layered pipeline with clear boundaries between components. Figure 4.1 shows the system context, illustrating how the QVM interacts with external actors (developers, students, administrators) and external systems (file system, web browsers, IBM Quantum platform). The system accepts quantum programs through three interfaces (CLI, Web API, Web GUI) and produces results in multiple formats (statevector, probabilities, visualizations, OpenQASM export).

The context diagram establishes the system boundary and identifies all external entities that interact with the QVM. Users provide quantum circuits as input and receive execution results as

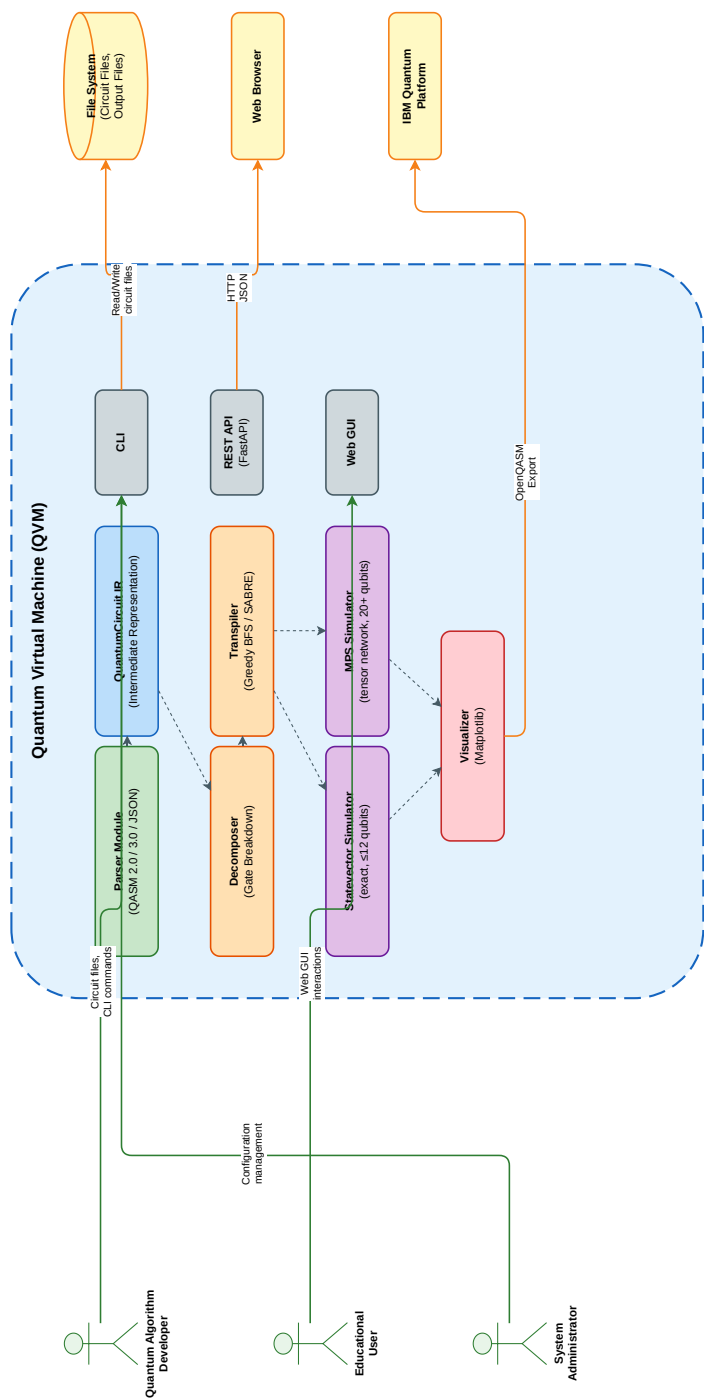


Figure 4.1: System Context Diagram

output. The system reads circuit files from the file system and can export results back to files. The web-based interfaces communicate with browsers via HTTP, and the OpenQASM export capability enables integration with external quantum platforms like IBM Quantum.

4.4 Architectural Design

The QVM follows a **7-layer pipeline architecture** as shown in Figure 4.2. Each layer represents a distinct phase in the quantum program lifecycle:

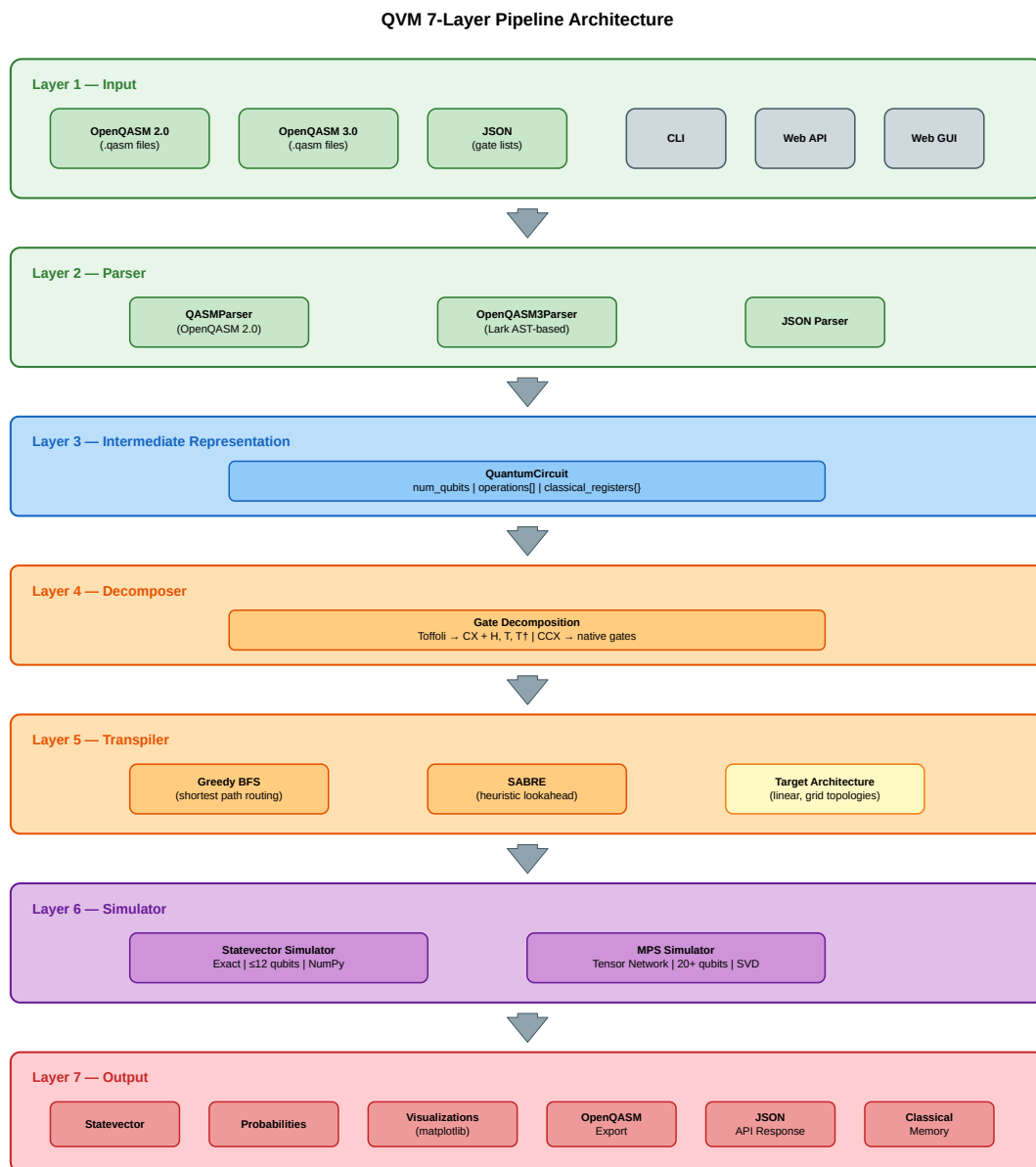


Figure 4.2: System Architecture Diagram — 7-Layer Pipeline

Layer 1 - Input Layer: Accepts quantum programs in three formats (OpenQASM 3.0, OpenQASM 2.0, JSON) through three interfaces (CLI, Web API, Web GUI). This layer provides flexibility in how users interact with the system.

Layer 2 - Parser Layer: Transforms textual circuit descriptions into structured Abstract Syntax Trees (AST). Uses Lark parser generator for OpenQASM 3.0 with formal grammar definition. Validates syntax and semantics before proceeding.

Layer 3 - Intermediate Representation (IR): Converts AST into a hardware-agnostic QuantumCircuit object. This layer decouples the input format from the execution backend, enabling the "Write Once, Run Anywhere" philosophy.

Layer 4 - Decomposer: Breaks down complex gates (Toffoli, controlled gates) into native gate sets supported by target architectures. Maps high-level operations to primitive gates (H, X, Y, Z, CX).

Layer 5 - Transpiler: Maps logical qubits to physical qubits according to hardware topology constraints. Implements two routing strategies (Greedy BFS, SABRE) and inserts SWAP gates to satisfy connectivity requirements.

Layer 6 - Simulator: Executes the transpiled circuit using either exact statevector simulation (up to 12 qubits) or efficient Matrix Product State (MPS) simulation (20+ qubits for low-entanglement circuits). Maintains classical memory for hybrid quantum-classical computation.

Layer 7 - Output Layer: Produces results in multiple formats: final statevector, measurement probabilities, circuit visualizations (matplotlib), and OpenQASM export for external tools.

This layered architecture ensures that each component has a single, well-defined responsibility and can be tested, modified, or replaced independently.

4.5 Activity Diagrams

Activity diagrams model the dynamic behavior of the system by showing the flow of control through various processes. The QVM system has three primary workflows:

4.5.1 Circuit Execution Flow

Figure 4.3 shows the complete workflow from loading a circuit file to displaying results. The process includes error handling for invalid files and syntax errors, backend selection (Statevector vs MPS), optional transpilation, and result visualization. The execution flow begins with the user providing a circuit file. The system validates the file existence and format, then invokes the appropriate parser. If parsing succeeds, the user can optionally request transpilation for a specific architecture. The system then selects the appropriate simulator based on circuit size and user preference. After simulation, results are displayed and optionally saved to files.

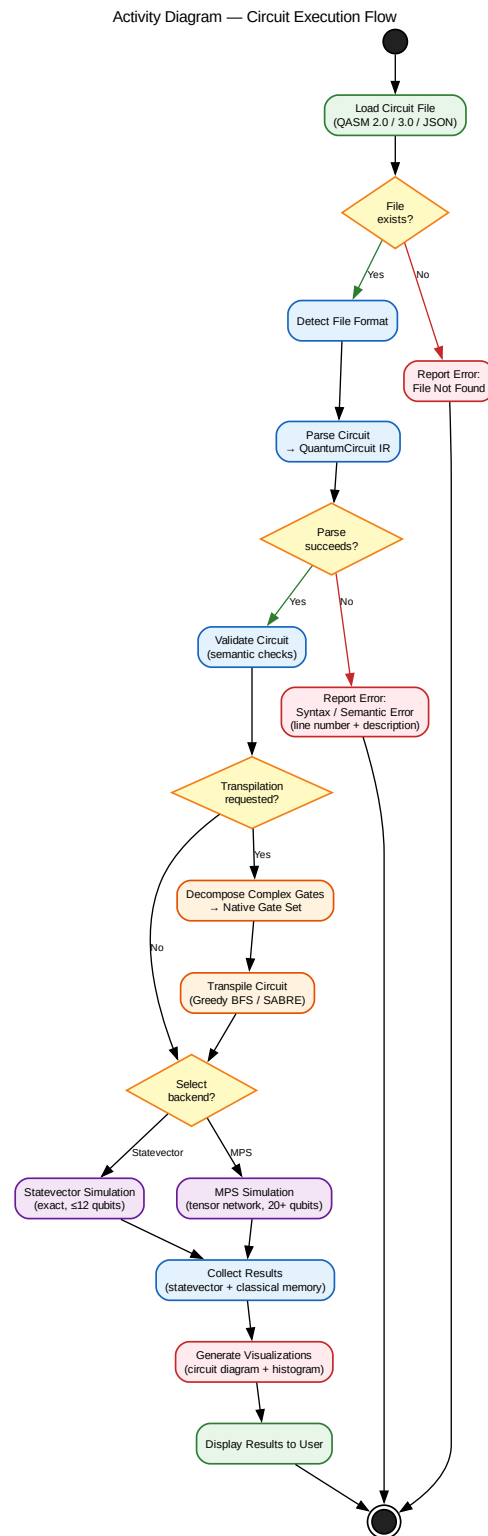


Figure 4.3: Activity Diagram — Circuit Execution Flow

4.5.2 Transpilation Process

Figure 4.4 details the transpilation workflow, showing how logical circuits are mapped to physical hardware topologies. The process includes gate decomposition, qubit routing (with choice between Greedy and SABRE algorithms), SWAP insertion, and optional mapping restoration. The transpilation process starts by analyzing the logical circuit's connectivity requirements. Complex gates are decomposed into native gates. The routing algorithm then determines the optimal qubit mapping, inserting SWAP gates where necessary to satisfy topology constraints. If mapping restoration is enabled, additional SWAPs are inserted to return qubits to their original logical positions.

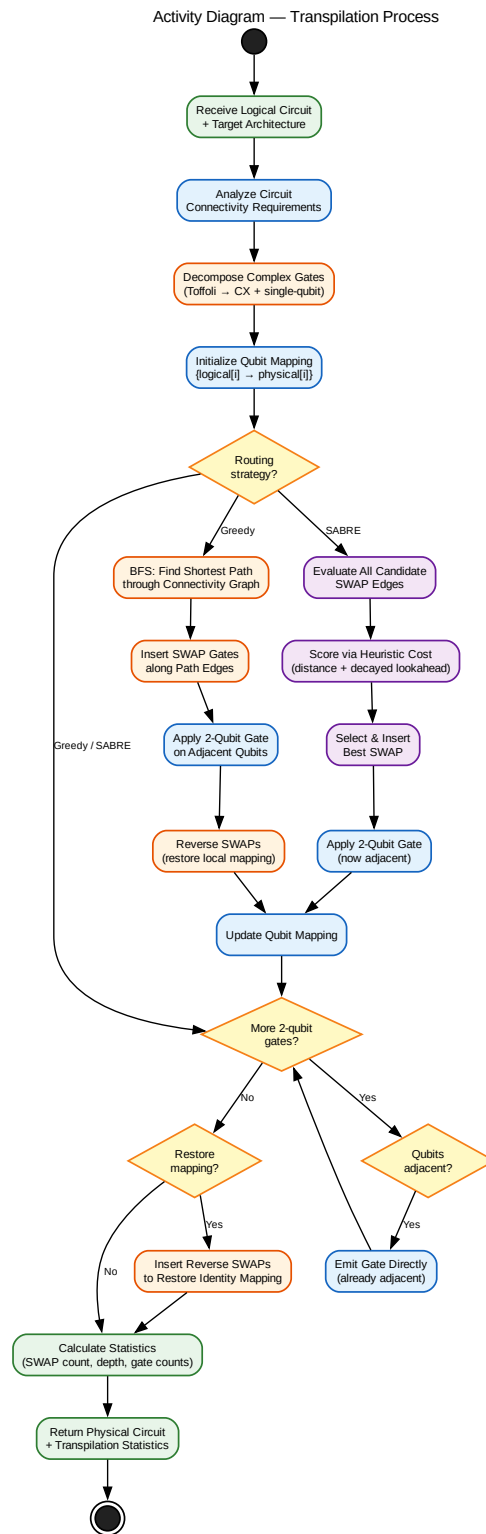


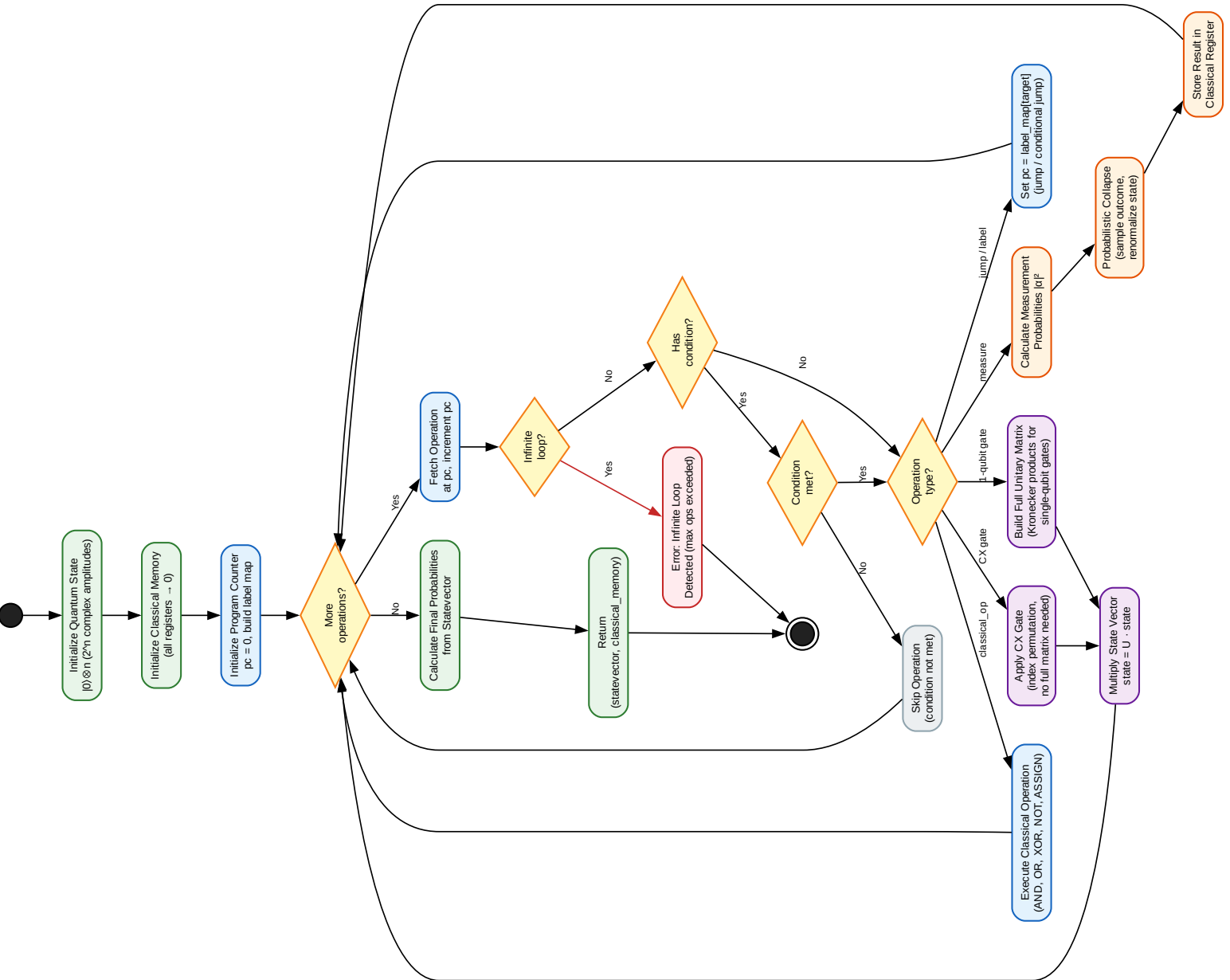
Figure 4.4: Activity Diagram — Transpilation Process

4.5.3 Simulation Process

Figure 4.5 illustrates the simulation workflow, including state initialization, gate application, measurement with collapse, and control flow handling (labels, jumps, conditionals). The simulation process initializes the quantum state to $|0\rangle^{\otimes n}$ and classical memory to zeros. It then iterates through circuit operations, applying unitary transformations for gates, performing probabilistic collapse for measurements, and updating classical memory. Control flow operations (if, while, jump) are handled by maintaining a program counter and label map. The process includes infinite loop detection to prevent runaway execution.

4.6 Class Diagram

The class diagram in Figure 4.6 shows the static structure of the QVM system, including all major classes, their attributes, methods, and relationships.



Quantum Virtual Machine — Class Diagram

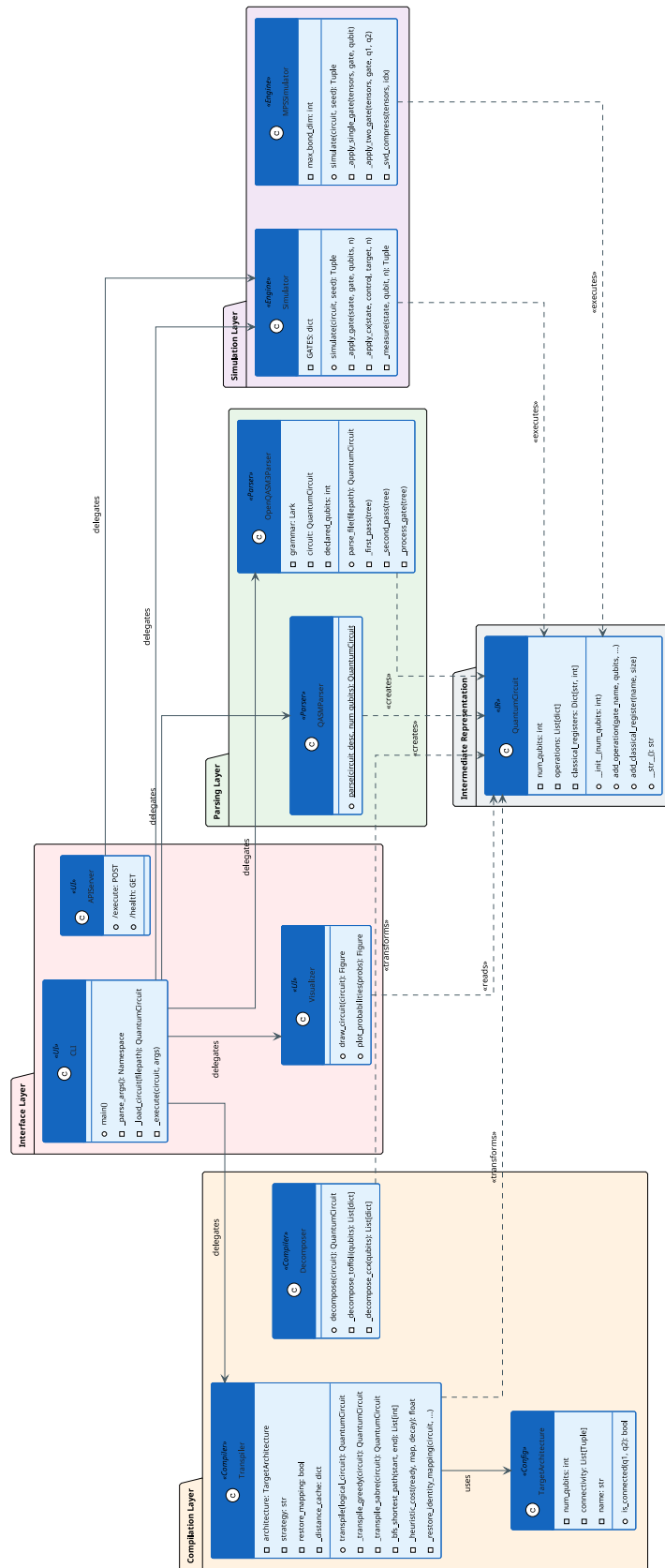


Figure 4.6: Class Diagram — System Structure

Core Classes:

QuantumCircuit: The central IR class that represents a hardware-agnostic quantum circuit. Contains `num_qubits`, `operations` list, and `classical_registers` dictionary. Provides methods for adding operations, classical registers, and string representation.

Simulator: Implements exact statevector simulation using NumPy. Contains gate matrices (H, X, Y, Z, CX, etc.) and methods for applying gates, measurements, and control flow. The `simulate()` method returns the final statevector and classical memory state.

MPSSimulator: Implements Matrix Product State simulation for efficient handling of low-entanglement circuits. Uses tensor contractions and SVD truncation to maintain computational efficiency.

Transpiler: Maps logical circuits to physical architectures. Contains `target_architecture`, `strategy` (greedy/sabre), and `restore_mapping` flag. Implements BFS routing, SABRE heuristics, and SWAP insertion algorithms.

OpenQASM3Parser: Parses OpenQASM 3.0 files using Lark grammar. Implements two-pass parsing: first pass for declarations, second pass for operations. Handles control flow (if, for, while) and classical operations.

QASMParser: Parses OpenQASM 2.0 files with basic gate support. Simpler than OpenQASM3Parser but sufficient for standard quantum algorithms.

TargetArchitecture: Represents hardware topology with `num_qubits` and `connectivity` list. Provides `is_connected()` method to check if two qubits are adjacent.

Decomposer: Breaks down complex gates into native gate sets. Implements decomposition rules (e.g., Toffoli $\rightarrow$ CNOTs + single-qubit gates).

Visualizer: Generates circuit diagrams and probability histograms using matplotlib. Provides `draw_circuit()` and `plot_probabilities()` methods.

CLI: Command-line interface that parses arguments and orchestrates the execution pipeline. Handles file loading, transpilation options, and result display.

APIServer: FastAPI-based REST API server. Provides `/execute` endpoint that accepts JSON circuit definitions and returns JSON results.

Relationships: - Simulator and MPSSimulator both depend on QuantumCircuit - Transpiler depends on QuantumCircuit and TargetArchitecture - Parsers create QuantumCircuit objects - CLI and APIServer orchestrate all components

4.7 Sequence Diagrams

Sequence diagrams show the dynamic interactions between objects over time. The QVM system has three key interaction scenarios:

4.7.1 CLI Execution Flow

Figure 4.7 shows the interaction sequence when a user executes a circuit via the command-line interface. The sequence begins with the user invoking the CLI with a circuit file. The CLI loads the file and delegates to the Parser, which creates a QuantumCircuit IR. If transpilation is requested, the CLI invokes the Transpiler with the target architecture. The Transpiler returns a physical circuit with SWAP gates inserted. The CLI then invokes the Simulator, which executes the

circuit and returns the statevector and classical memory. Finally, the CLI invokes the Visualizer to display results to the user.

Sequence Diagram — CLI Execution Flow

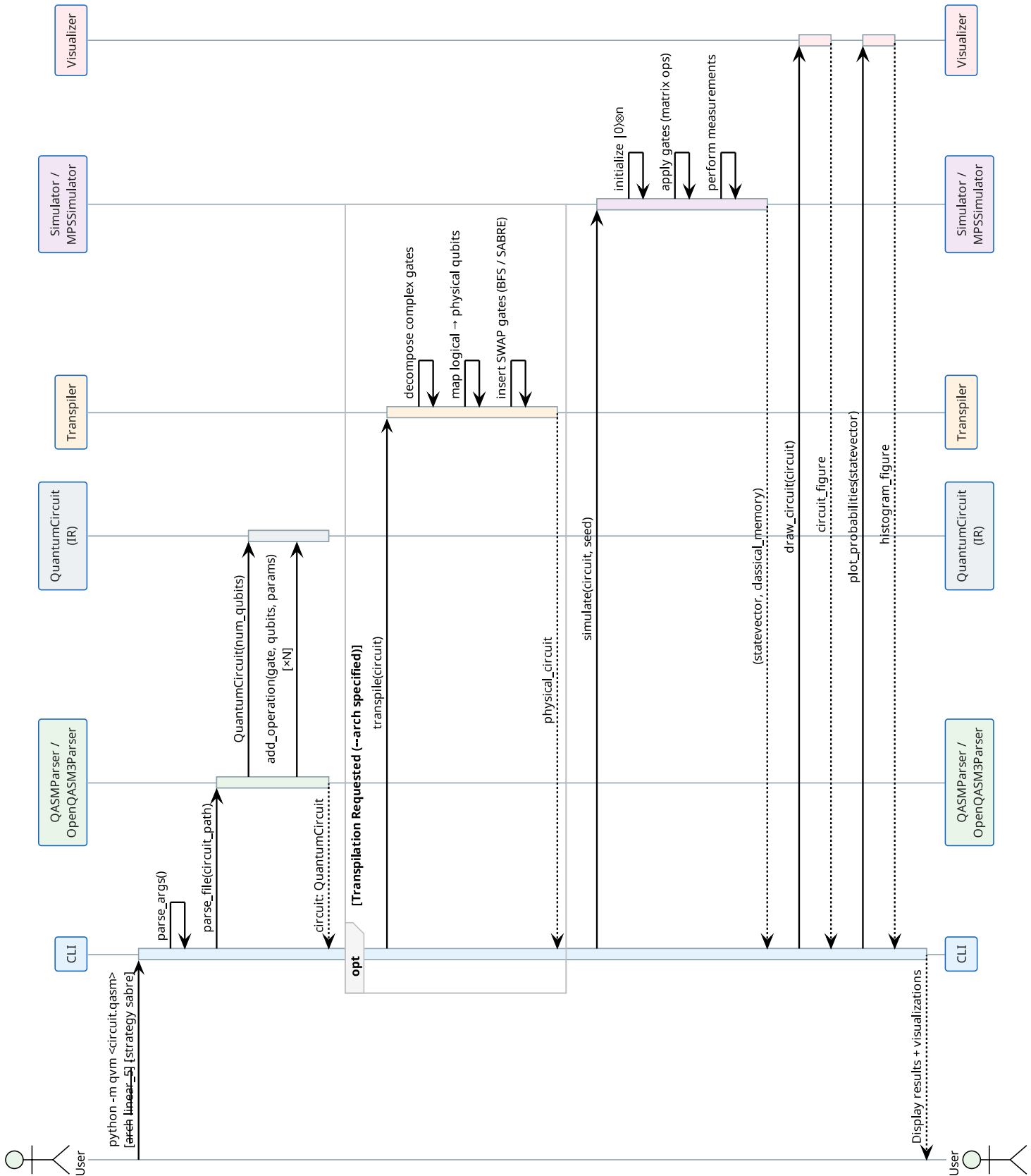


Figure 4.7: Sequence Diagram — CLI Execution Flow

4.7.2 API Request Flow

Figure 4.8 illustrates the interaction when a client submits a circuit via the REST API. The client sends an HTTP POST request with a JSON circuit definition. The APIServer validates the request format and extracts the circuit data. It then delegates to the Parser to create a QuantumCircuit IR. The APIServer invokes the Simulator to execute the circuit. After receiving the results, the APIServer formats them as JSON and returns an HTTP 200 response to the client. Error handling is included for invalid JSON (HTTP 400) and execution errors (HTTP 500).

Sequence Diagram — REST API Request Flow

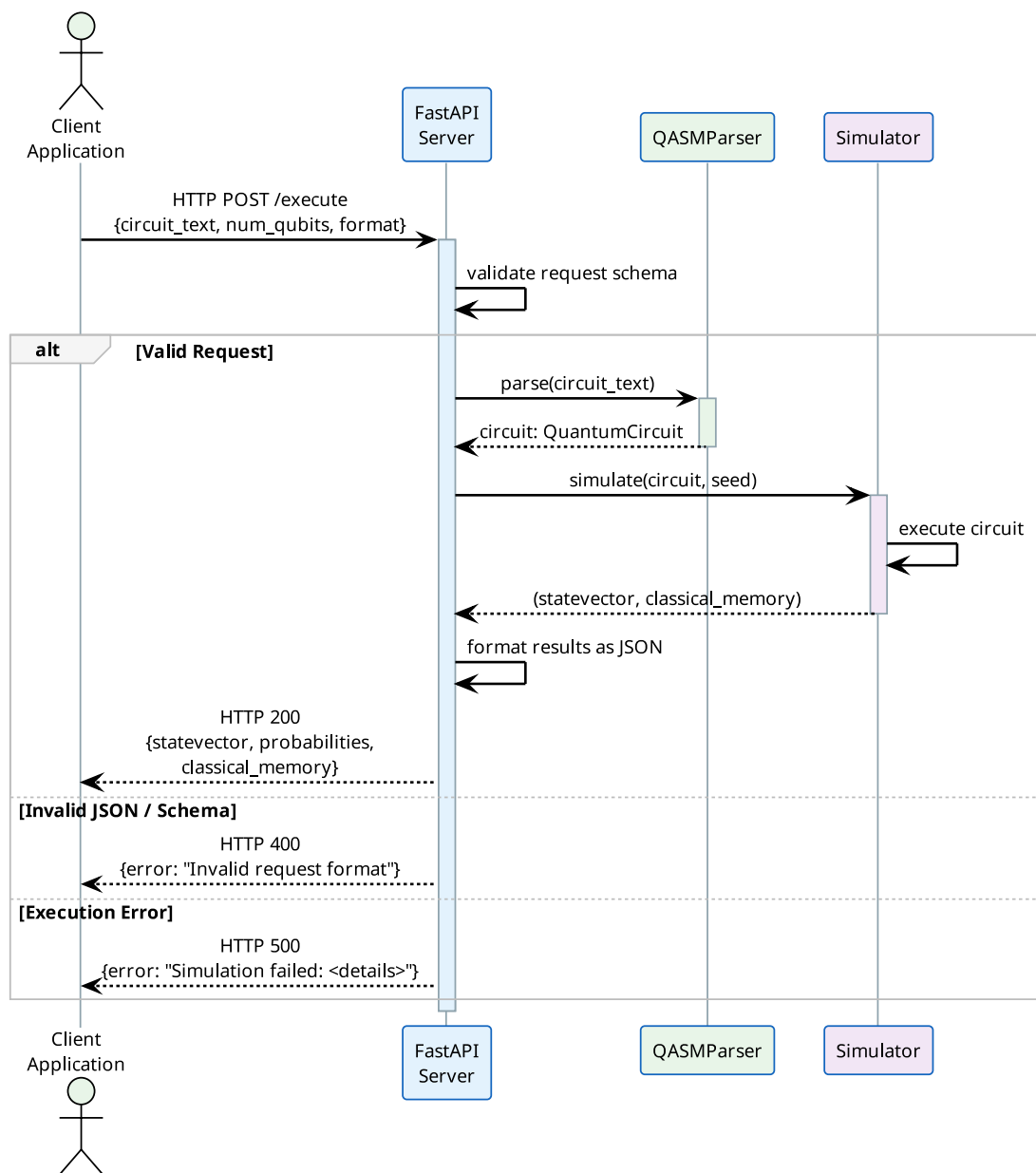


Figure 4.8: Sequence Diagram — REST API Request Flow

4.7.3 Transpilation Detail

Figure 4.9 shows the detailed interaction during the transpilation process. The Transpiler receives a logical circuit and target architecture. It first invokes the Decomposer to break down complex gates. The Decomposer returns a circuit with only native gates. The Transpiler then analyzes connectivity requirements and selects the routing strategy (Greedy or SABRE). For each two-qubit gate that violates topology constraints, the Transpiler inserts SWAP gates to bring qubits into adjacency. The process continues until all gates satisfy connectivity requirements. If mapping restoration is enabled, additional SWAPs are inserted to restore the original logical-to-physical mapping.

Sequence Diagram — Transpilation Detail

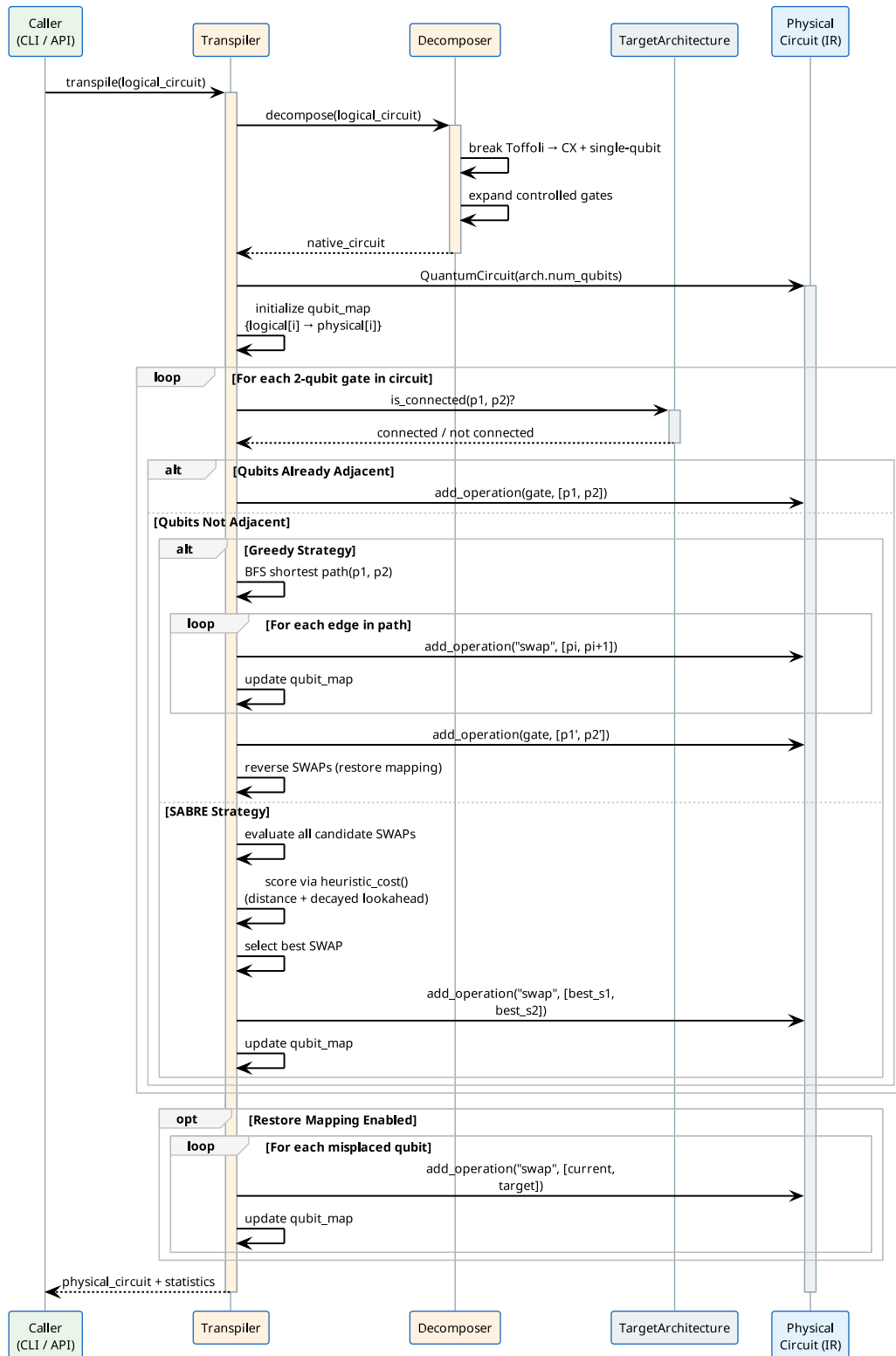


Figure 4.9: Sequence Diagram — Transpilation Detail

4.8 State Transition Diagrams

State transition diagrams model the lifecycle of key entities in the system. Figure 4.10 shows the state transitions for a quantum circuit as it progresses through the QVM pipeline.

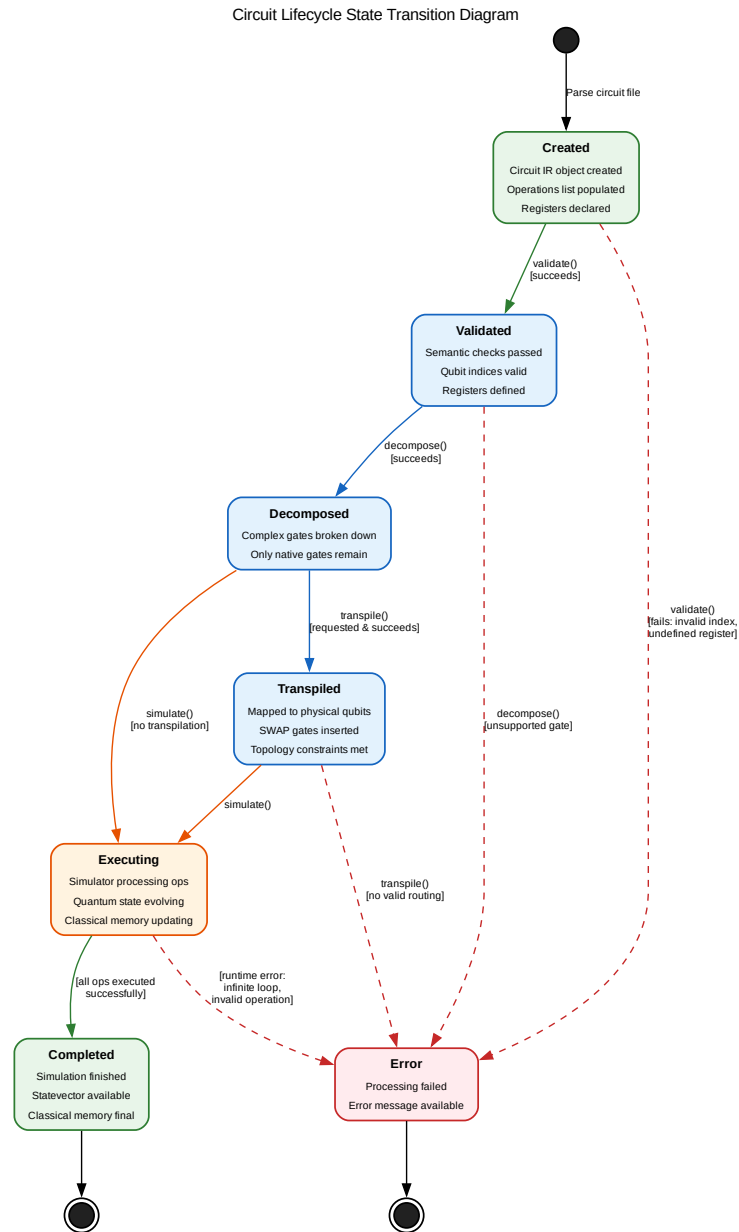


Figure 4.10: State Transition Diagram — Circuit Lifecycle

Circuit States:

Created: Initial state after parsing. The circuit exists as a QuantumCircuit IR object but has not been validated or processed.

Validated: The circuit has passed semantic validation (qubit indices in range, valid gate parameters, classical registers declared).

Decomposed: Complex gates have been broken down into native gate sets. The circuit now contains only primitive operations.

Transpiled: The circuit has been mapped to a physical architecture with SWAP gates inserted. Logical qubits are mapped to physical qubits.

Executing: The simulator is actively processing the circuit operations. The quantum state is being evolved step-by-step.

Completed: Simulation has finished successfully. The final statevector and classical memory are available.

Error: An error occurred during any stage (parsing, validation, transpilation, or execution). The circuit cannot proceed further.

State Transitions: - Created → Validated: Semantic validation succeeds - Created → Error: Validation fails (invalid qubit index, undefined register) - Validated → Decomposed: Gate decomposition succeeds - Decomposed → Transpiled: Transpilation succeeds (or skipped if not requested) - Transpiled → Executing: Simulation begins - Executing → Completed: All operations executed successfully - Executing → Error: Runtime error (infinite loop, invalid operation)

4.9 Data Flow Diagrams

4.9.1 Level 1 DFD - System Level

Figure 4.11 shows the high-level data flows between the QVM system and external entities.

External Entities: - User: Provides circuit files and execution parameters - File System: Stores circuit files and output results - Web Browser: Displays web GUI and receives results

Data Flows: - Circuit File → QVM System: User provides quantum program - Execution Parameters → QVM System: User specifies options (architecture, simulator, seed) - Results → User: System returns statevector, probabilities, visualizations - Circuit File ← File System: System reads input files - Output Files → File System: System writes visualizations and exports - HTTP Request → QVM System: Browser sends circuit via API - HTTP Response → Web Browser: System returns JSON results

Data Stores: - Circuit Files: Persistent storage of quantum programs - Configuration: System settings and architecture definitions

4.9.2 Level 2 DFD - Module Level

Figure 4.12 shows the detailed data flows between internal modules.

Processes:

P1 - Parse Circuit: Accepts circuit text and produces QuantumCircuit IR. Validates syntax and semantics.

P2 - Decompose Gates: Accepts QuantumCircuit with complex gates and produces QuantumCircuit with native gates only.

P3 - Transpile Circuit: Accepts logical QuantumCircuit and target architecture, produces physical QuantumCircuit with SWAP gates.

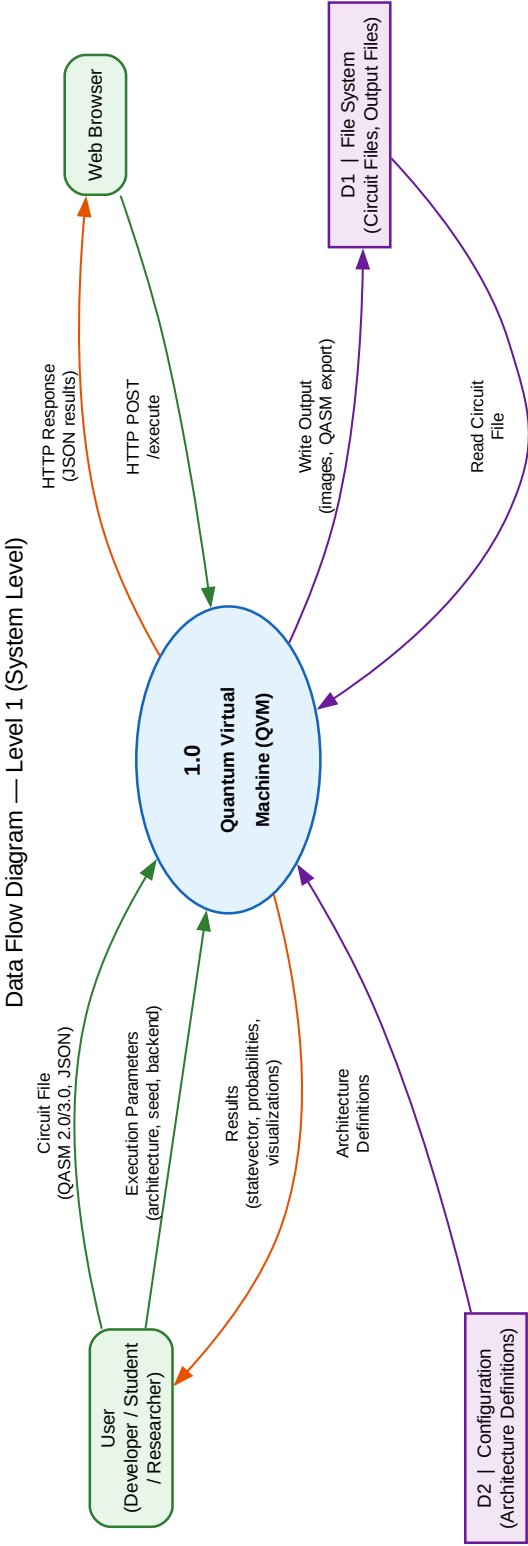


Figure 4.11: Data Flow Diagram — Level 1 (System Level)

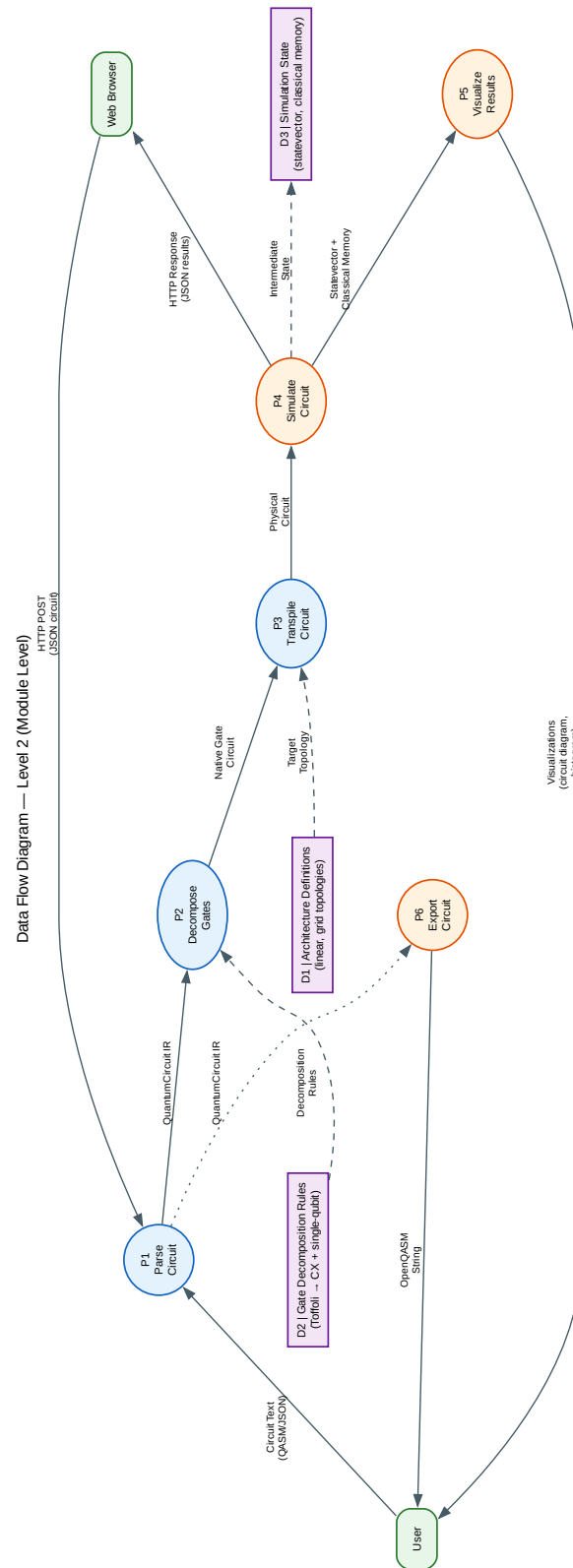


Figure 4.12: Data Flow Diagram — Level 2 (Module Level)

P4 - Simulate Circuit: Accepts QuantumCircuit and execution parameters, produces statevector and classical memory.

P5 - Visualize Results: Accepts statevector and circuit, produces diagrams and histograms.

P6 - Export Circuit: Accepts QuantumCircuit, produces OpenQASM string.

Data Flows: - Circuit Text → P1: Raw input from file or API - QuantumCircuit IR → P2: Parsed circuit with all gates - Native Gate Circuit → P3: Circuit with only primitive gates - Physical Circuit → P4: Transpiled circuit ready for execution - Statevector + Classical Memory → P5: Simulation results - QuantumCircuit IR → P6: Circuit to be exported - OpenQASM String → User: Exported circuit for external tools

Data Stores: - Architecture Definitions: Topology specifications (linear, grid) - Gate Decomposition Rules: Mapping from complex gates to native gates - Simulation State: Intermediate quantum state during execution

4.10 Entity-Relationship Diagram

The QVM system does not use a traditional relational database. Instead, it uses a **file-based data model** where quantum circuits are stored as text files (OpenQASM, JSON) and results are stored as output files (PNG images, QASM exports).

Rationale for File-Based Approach:

- **Simplicity:** Quantum circuits are naturally represented as text files, making them human-readable and version-controllable.
- **Portability:** File-based storage allows easy sharing of circuits between users and systems without database dependencies.
- **Stateless Execution:** Each circuit execution is independent, requiring no persistent state between runs.
- **Educational Focus:** File-based approach is easier for students to understand and debug compared to database abstractions.

File Structure:

- **Input Files:** .qasm (OpenQASM 2.0/3.0), .json (JSON gate lists)
- **Output Files:** .png (visualizations), .qasm (exports), .txt (results)
- **Configuration Files:** .json (architecture definitions)

If the system were to be extended with a database for storing execution history or user accounts, a simple schema would include:

- **Users:** user_id, username, email
- **Circuits:** circuit_id, user_id, name, qasm_text, created_at
- **Executions:** execution_id, circuit_id, architecture, simulator, results, timestamp

However, this is beyond the current scope of the educational QVM system.

4.11 Data Dictionary

The data dictionary documents the structure of key data elements in the QVM system.

4.11.1 QuantumCircuit IR Structure

This section defines the internal representation for the quantum circuit, storing the number of qubits and sequences of operations.

Table 4.1: QuantumCircuit Data Structure

Attribute	Data Type	Description
num_qubits	int	Number of qubits in the circuit (must be positive integer)
operations	list[dict]	Ordered list of quantum and classical operations
classical_registers	dict[str, int]	Mapping from register name to size

4.11.2 Operation Dictionary Structure

The operation dictionary specifies each individual quantum or classical operation within a circuit.

Table 4.2: Operation Data Structure

Field	Data Type	Description
name	str	Gate name (h, x, cx, measure, etc.)
qubits	list[int]	Target qubit indices (0-indexed)
params	list[float]	Gate parameters (e.g., rotation angles)
condition	dict — None	Conditional execution specification
target_bit	tuple — None	(register_name, index) for measurement results
duration	str — None	Timing specification (e.g., "100ns")
label	str — None	Label name for control flow
jump_to	str — None	Target label for jump operations
classical_op	dict — None	Classical operation specification

4.11.3 Condition Dictionary Structure

The condition structure defines the conditions under which an operation should be conditionally executed based on classical bits.

Table 4.3: Condition Data Structure

Field	Data Type	Description
register	str	Classical register name
index	int	Bit index within register
value	int	Expected value (0 or 1) for condition to be true

4.11.4 Classical Operation Dictionary Structure

Classical operations define operations performed on classical registers such as logical AND, OR, and XOR.

Table 4.4: Classical Operation Data Structure

Field	Data Type	Description
op	str	Operator (=, &, —, ^, ~)
target	tuple	(register_name, index) for result storage
args	list	Operands (tuples for register references, ints for literals)

4.11.5 Target Architecture Structure

This structure represents the physical architecture of the hardware, including its qubit connectivity constraints.

Table 4.5: TargetArchitecture Data Structure

Attribute	Data Type	Description
num_qubits	int	Total number of physical qubits
connectivity	list[tuple]	List of (qubit1, qubit2) pairs representing edges
name	str	Architecture name (e.g., "linear_5", "grid_3x3")

4.11.6 Simulation Results Structure

The simulation results capture the final execution outcomes, including the statevector and measurement probabilities.

Table 4.6: Simulation Results Data Structure

Field	Data Type	Description
statevector	ndarray	Complex-valued array of size 2^n representing quantum state
classical_memory	dict[str, ndarray]	Mapping from register name to integer array of bit values
probabilities	dict[str, float]	Mapping from basis state to measurement probability
execution_time	float	Time taken for simulation (seconds)

4.12 Summary

This chapter has presented a comprehensive software design for the Quantum Virtual Machine system. The design follows object-oriented principles with a clear 7-layer pipeline architecture that separates concerns and enables modularity. The system structure was documented through multiple UML perspectives: context diagram (system boundary), architecture diagram (layered pipeline), class diagram (static structure), activity diagrams (behavioral workflows), sequence diagrams (object interactions), state diagram (circuit lifecycle), and data flow diagrams (information flow). The design emphasizes hardware abstraction through the QuantumCircuit IR, flexibility through the Strategy pattern for transpilation, and extensibility through modular components. The file-based data model provides simplicity and portability suitable for an educational quantum computing platform. The data dictionary documents the structure of all key data elements, ensuring clear understanding of the system's information architecture. This design provides a solid foundation for the implementation presented in the next chapter.